

```
# Individual Support Module
## Physics-Grounded Personal System Dynamics
```

```
---
```

```
## Module Structure
```

```
```python
```

```
individual_support_module.py
A physics-inspired framework for modeling and supporting personal system dynamics
"""
```

```
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple
from enum import Enum
import numpy as np
from datetime import datetime, timedelta
```

```
=====
CORE STATE PRIMITIVES
=====
```

```
@dataclass
class SystemState:
```

```
 """
 Core state vector for personal system dynamics
 Maps personal primitives to physics observables
 """
```

```
 timestamp: datetime
```

```
 # Primary state variables
 pressure: float # P(t) - Force per unit capacity [0-10]
 margin: float # M(t) - Distance to failure [0-10]
 temperature: float # T(t) - Internal agitation [0-10]
 drift: float # D(t) - Cumulative displacement [-5 to +5]
 escalation_risk: float # E(t) - Proximity to critical point [0-1]
```

```
 # Derived metrics
 strain: float = field(init=False) # Overall system strain
 fragility: float = field(init=False) # Inverse of margin
```

```
 # Uncertainty measures
 measurement_noise: Dict[str, float] = field(default_factory=dict)
 observability: float = 0.7 # How much of state is visible [0-1]
```

```
 def __post_init__(self):
 """Calculate derived quantities"""
 self.strain = self._calculate_strain()
 self.fragility = 10.0 - self.margin
```

```
 def _calculate_strain(self) -> float:
```

```
 """
 Universal strain function
 Strain = f(P, M, T, D, E)
 """
```

```

Nonlinear coupling between state variables
base_strain = (
 0.3 * self.pressure +
 0.25 * self.fragility +
 0.2 * self.temperature +
 0.15 * abs(self.drift) +
 0.1 * (self.escalation_risk * 10)
)

Amplification when margin is low and pressure is high
margin_multiplier = 1.0 + (self.pressure * self.fragility / 100)

return min(10.0, base_strain * margin_multiplier)

=====
PHYSICS ANALOGUES
=====

class PhysicsMapping:
 """
 Explicit mappings between personal and physical domains
 """

 @staticmethod
 def pressure_to_force_density(
 demands: List[float],
 capacity_area: float
) -> float:
 """
 $P = F/A$
 Pressure = Total demands / Available capacity
 """
 total_force = sum(demands)
 return total_force / max(capacity_area, 0.1)

 @staticmethod
 def safety_factor(
 current_load: float,
 failure_threshold: float
) -> float:
 """
 Margin = failure_load / working_load
 Higher is better
 """
 return failure_threshold / max(current_load, 0.1)

 @staticmethod
 def thermal_agitation(
 emotional_variance: float,
 baseline_volatility: float = 1.0
) -> float:
 """
 Temperature ~ average kinetic energy of internal states
 Higher variance = higher temperature
 """
 return emotional_variance / baseline_volatility

```

```

@staticmethod
def drift_integral(
 small_imbalances: List[float],
 time_delta: float
) -> float:
 """
 Drift = $\int$ imbalances dt
 Cumulative deviation from equilibrium
 """
 return np.trapz(small_imbalances, dx=time_delta)

```

```

@staticmethod
def phase_transition_probability(
 current_state: float,
 critical_point: float,
 temperature: float
) -> float:
 """
 P(transition) increases near critical points
 Temperature adds noise that can trigger transition
 """
 distance = abs(current_state - critical_point)
 # Boltzmann-like probability
 return np.exp(-distance / max(temperature, 0.1))

```

```

=====
DYNAMICS ENGINE
=====

```

```

@dataclass
class SystemDynamics:
 """
 State-space model of personal system evolution
 dx/dt = f(x, u, t)
 """

 # State history
 trajectory: List[SystemState] = field(default_factory=list)

 # System parameters
 damping_coefficient: float = 0.3 # Natural decay rate
 noise_level: float = 0.1 # Stochastic forcing

 def evolve(
 self,
 current_state: SystemState,
 control_input: 'ControlInput',
 dt: float = 1.0
) -> SystemState:
 """
 Time evolution of system state
 """
 # Unpack current state
 P, M, T, D, E = (
 current_state.pressure,

```

```

 current_state.margin,
 current_state.temperature,
 current_state.drift,
 current_state.escalation_risk
)

 # State evolution equations (Euler integration)

 # Pressure dynamics: external forcing - control - damping
 dP = (
 control_input.external_load_change
 - control_input.pressure_relief
 - self.damping_coefficient * P
)
 P_new = np.clip(P + dP * dt, 0, 10)

 # Margin dynamics: capacity building - erosion
 dM = (
 control_input.capacity_building
 - control_input.margin_erosion
 + 0.1 * (10 - P) # Margin recovers when pressure is low
)
 M_new = np.clip(M + dM * dt, 0, 10)

 # Temperature dynamics: agitation - cooling
 dT = (
 0.2 * P # Pressure increases temperature
 - control_input.cooling_input
 - 0.2 * T # Natural cooling
)
 T_new = np.clip(T + dT * dt, 0, 10)

 # Drift dynamics: cumulative imbalance
 dD = control_input.realignment_force - 0.1 * D
 D_new = np.clip(D + dD * dt, -5, 5)

 # Escalation risk: function of pressure, margin, and temperature
 E_new = PhysicsMapping.phase_transition_probability(
 current_state=P_new,
 critical_point=8.0,
 temperature=T_new
)

 # Add stochastic noise
 noise = np.random.normal(0, self.noise_level, 5)

 new_state = SystemState(
 timestamp=current_state.timestamp + timedelta(hours=dt),
 pressure=P_new + noise[0],
 margin=M_new + noise[1],
 temperature=T_new + noise[2],
 drift=D_new + noise[3],
 escalation_risk=np.clip(E_new + noise[4], 0, 1)
)

 self.trajectory.append(new_state)
 return new_state

```

```

def predict_trajectory(
 self,
 current_state: SystemState,
 control_sequence: List['ControlInput'],
 horizon: int = 24
) -> List[SystemState]:
 """
 Forward simulation of system trajectory
 Model Predictive Control (MPC) style
 """
 predicted = [current_state]
 state = current_state

 for i in range(horizon):
 control = control_sequence[i] if i < len(control_sequence) else ControlInput()
 state = self.evolve(state, control, dt=1.0)
 predicted.append(state)

 return predicted

=====
CONTROL SYSTEM
=====

@dataclass
class ControlInput:
 """
 Control inputs that can modify system trajectory
 Small, well-placed inputs = high leverage
 """
 # Pressure controls
 pressure_relief: float = 0.0 # Remove or delegate demands
 external_load_change: float = 0.0 # New demands appearing

 # Margin controls
 capacity_building: float = 0.0 # Increase resources/skills
 margin_erosion: float = 0.0 # Depletion of reserves

 # Temperature controls
 cooling_input: float = 0.0 # Calming interventions

 # Drift controls
 realignment_force: float = 0.0 # Corrective action

 # Metadata
 leverage: float = 0.0 # Expected ΔStrain per unit effort
 confidence: float = 0.5 # Certainty about control effectiveness

class ControlStrategy(Enum):
 """Control strategies mapped to physical principles"""
 PRESSURE_RELIEF = "Remove load from system"
 MARGIN_EXPANSION = "Increase safety factor"
 THERMAL_REGULATION = "Reduce agitation/volatility"
 DRIFT_CORRECTION = "Realign with equilibrium"

```

```
ESCALATION_PREVENTION = "Move away from critical point"
TRAJECTORY_OPTIMIZATION = "Optimal path planning"
```

```
@dataclass
class Controller:
```

```
 """
 Stabilizing controller for personal systems
 Acts like ClarityOS autopilot
 """
```

```
def compute_optimal_control(
 self,
 current_state: SystemState,
 target_state: SystemState,
 constraints: Dict[str, float]
) -> ControlInput:
 """
 Compute control input that moves system toward target
 Subject to constraints on available actions
 """

 # Error vector
 error_pressure = target_state.pressure - current_state.pressure
 error_margin = target_state.margin - current_state.margin
 error_temp = target_state.temperature - current_state.temperature
 error_drift = target_state.drift - current_state.drift

 # Proportional control gains (tunable)
 Kp_pressure = 0.5
 Kp_margin = 0.3
 Kp_temp = 0.4
 Kp_drift = 0.2

 # Compute control signals
 control = ControlInput(
 pressure_relief=max(0, Kp_pressure * error_pressure),
 capacity_building=max(0, Kp_margin * error_margin),
 cooling_input=max(0, Kp_temp * error_temp),
 realignment_force=Kp_drift * error_drift
)

 # Apply constraints
 control = self._apply_constraints(control, constraints)

 # Estimate leverage
 control.leverage = self._estimate_leverage(current_state, control)

 return control

def _apply_constraints(
 self,
 control: ControlInput,
 constraints: Dict[str, float]
) -> ControlInput:
 """Apply physical/practical limits to control inputs"""
 max_relief = constraints.get('max_pressure_relief', 2.0)
 max_building = constraints.get('max_capacity_building', 1.0)
```

```

max_cooling = constraints.get('max_cooling', 1.5)

control.pressure_relief = min(control.pressure_relief, max_relief)
control.capacity_building = min(control.capacity_building, max_building)
control.cooling_input = min(control.cooling_input, max_cooling)

return control

def _estimate_leverage(
 self,
 state: SystemState,
 control: ControlInput
) -> float:
 """
 Estimate mechanical advantage:
 How much strain reduction per unit control effort?
 """
 total_effort = (
 abs(control.pressure_relief) +
 abs(control.capacity_building) +
 abs(control.cooling_input) +
 abs(control.realignment_force)
)

 if total_effort < 0.01:
 return 0.0

 # Expected strain reduction (linearized)
 expected_delta_strain = (
 0.3 * control.pressure_relief +
 0.25 * control.capacity_building +
 0.2 * control.cooling_input +
 0.15 * abs(control.realignment_force)
)

 return expected_delta_strain / total_effort

def emergency_stabilization(
 self,
 critical_state: SystemState
) -> List[ControlInput]:
 """
 Emergency protocol when system near critical point
 Maximum intervention to prevent phase transition
 """
 interventions = []

 # Immediate pressure relief
 if critical_state.pressure > 8.0:
 interventions.append(ControlInput(
 pressure_relief=3.0,
 leverage=5.0, # High-leverage emergency action
 confidence=0.8
))

 # Rapid cooling if temperature critical
 if critical_state.temperature > 8.0:

```

```

 interventions.append(ControlInput(
 cooling_input=2.5,
 leverage=4.0,
 confidence=0.7
))

Margin expansion if fragile
if critical_state.margin < 2.0:
 interventions.append(ControlInput(
 capacity_building=1.5,
 margin_erosion=-0.5, # Stop depletion
 leverage=3.5,
 confidence=0.6
))

return interventions

=====
OBSERVATION & INFERENCE
=====

@dataclass
class Observer:
 """
 Handles partial observability and measurement noise
 Kalman filter-like state estimation
 """

 # Prior belief about state
 belief_mean: np.ndarray = field(default_factory=lambda: np.zeros(5))
 belief_covariance: np.ndarray = field(default_factory=lambda: np.eye(5))

 def observe(
 self,
 noisy_measurements: Dict[str, float],
 measurement_confidence: Dict[str, float]
) -> SystemState:
 """
 Infer true state from noisy, partial measurements
 """
 # Measurement vector
 z = np.array([
 noisy_measurements.get('pressure', 5.0),
 noisy_measurements.get('margin', 5.0),
 noisy_measurements.get('temperature', 5.0),
 noisy_measurements.get('drift', 0.0),
 noisy_measurements.get('escalation_risk', 0.3)
])

 # Measurement noise covariance
 R = np.diag([
 1.0 / measurement_confidence.get('pressure', 0.5),
 1.0 / measurement_confidence.get('margin', 0.5),
 1.0 / measurement_confidence.get('temperature', 0.5),
 1.0 / measurement_confidence.get('drift', 0.5),
 1.0 / measurement_confidence.get('escalation_risk', 0.3)
])

```



```

])

Simple Bayesian update (simplified Kalman)
K = self.belief_covariance @ np.linalg.inv(
 self.belief_covariance + R
)
self.belief_mean = self.belief_mean + K @ (z - self.belief_mean)
self.belief_covariance = (np.eye(5) - K) @ self.belief_covariance

Construct state from belief
return SystemState(
 timestamp=datetime.now(),
 pressure=float(self.belief_mean[0]),
 margin=float(self.belief_mean[1]),
 temperature=float(self.belief_mean[2]),
 drift=float(self.belief_mean[3]),
 escalation_risk=float(self.belief_mean[4]),
 measurement_noise={
 k: float(np.sqrt(self.belief_covariance[i, i]))
 for i, k in enumerate(['pressure', 'margin', 'temperature', 'drift', 'escalation_risk'])
 }
)

```

```

=====
WITNESSED CLARITY
=====

```

@dataclass

class ReferenceFrame:

"""

Stable, non-judgmental coordinate system  
 Makes dynamics interpretable and actionable  
 """

name: str

origin: str # What defines "equilibrium" in this frame

basis\_vectors: Dict[str, str] # How to interpret each dimension

gauge\_choice: str # What's held constant to make sense of change

def transform\_to\_frame(

self,

raw\_state: SystemState

) -> Dict[str, str]:

"""

Express system state in this reference frame  
 Returns human-interpretable narrative  
 """

```

interpretation = {
 'frame': self.name,
 'narrative': []
}

```

# Interpret pressure

if raw\_state.pressure > 7:

interpretation['narrative'].append(

f" High load: You're carrying {raw\_state.pressure:.1f}/10 demands relative to your

{self.basis\_vectors['pressure']}")

```

)
elif raw_state.pressure < 3:
 interpretation['narrative'].append(
 f" Sustainable load: Current demands are well within capacity"
)

Interpret margin
if raw_state.margin < 3:
 interpretation['narrative'].append(
 f" Low margin: Only {raw_state.margin:.1f}/10 buffer before {self.basis_vectors['margin']}"
)

Interpret temperature
if raw_state.temperature > 7:
 interpretation['narrative'].append(
 f" High agitation: Internal state is volatile ({raw_state.temperature:.1f}/10)"
)

Interpret drift
if abs(raw_state.drift) > 2:
 direction = "away from" if raw_state.drift > 0 else "toward"
 interpretation['narrative'].append(
 f" Significant drift: Slowly moving {direction} {self.origin}"
)

Interpret escalation risk
if raw_state.escalation_risk > 0.6:
 interpretation['narrative'].append(
 f" Critical proximity: {raw_state.escalation_risk:.0%} probability of phase transition"
)

return interpretation

```

```

=====
SUPPORT INTERVENTIONS
=====

```

```

class InterventionLibrary:

```

```

 """
 Catalog of high-leverage interventions
 Mapped to control strategies
 """

```

```

 @staticmethod

```

```

 def pressure_relief_interventions() -> List[Dict]:
 return [
 {
 'name': 'Delegation',
 'control': ControlInput(pressure_relief=2.0),
 'leverage': 4.5,
 'time_to_effect': 'hours',
 'description': 'Transfer specific demands to others'
 },
 {
 'name': 'Postponement',
 'control': ControlInput(pressure_relief=1.5),

```

```

 'leverage': 3.0,
 'time_to_effect': 'minutes',
 'description': 'Move non-urgent demands out of critical path'
 },
 {
 'name': 'Elimination',
 'control': ControlInput(pressure_relief=3.0),
 'leverage': 5.0,
 'time_to_effect': 'immediate',
 'description': 'Remove demands entirely (say no)'
 }
]

```

```

@staticmethod
def margin_expansion_interventions() -> List[Dict]:
 return [
 {
 'name': 'Skill Acquisition',
 'control': ControlInput(capacity_building=1.5),
 'leverage': 3.5,
 'time_to_effect': 'weeks',
 'description': 'Increase throughput capacity via learning'
 },
 {
 'name': 'Resource Injection',
 'control': ControlInput(capacity_building=2.0, margin_erosion=-1.0),
 'leverage': 4.0,
 'time_to_effect': 'days',
 'description': 'Add time, money, or people to system'
 },
 {
 'name': 'Process Optimization',
 'control': ControlInput(capacity_building=1.0),
 'leverage': 3.0,
 'time_to_effect': 'days',
 'description': 'Reduce waste in existing workflows'
 }
]

```

```

@staticmethod
def thermal_regulation_interventions() -> List[Dict]:
 return [
 {
 'name': 'Physiological Cooling',
 'control': ControlInput(cooling_input=2.0),
 'leverage': 4.0,
 'time_to_effect': 'minutes',
 'description': 'Breathing, movement, sensory regulation'
 },
 {
 'name': 'Cognitive Reframe',
 'control': ControlInput(cooling_input=1.5),
 'leverage': 3.5,
 'time_to_effect': 'hours',
 'description': 'Change interpretation of stressors'
 }
]

```

```

 'name': 'Environmental Modification',
 'control': ControlInput(cooling_input=1.0),
 'leverage': 2.5,
 'time_to_effect': 'immediate',
 'description': 'Remove triggering stimuli from environment'
 }
]

```

```

=====
MAIN SUPPORT INTERFACE
=====

```

```

class IndividualSupportModule:

```

```

 """

```

```

 Main interface for physics-grounded personal support

```

```

 """

```

```

 def __init__(self):

```

```

 self.dynamics = SystemDynamics()

```

```

 self.controller = Controller()

```

```

 self.observer = Observer()

```

```

 self.interventions = InterventionLibrary()

```

```

 # Default reference frame

```

```

 self.default_frame = ReferenceFrame(

```

```

 name="Sustainable Equilibrium",

```

```

 origin="State where demands match capacity with 30% margin",

```

```

 basis_vectors={

```

```

 'pressure': 'total available capacity',

```

```

 'margin': 'structural failure',

```

```

 'temperature': 'baseline emotional volatility',

```

```

 'drift': 'long-term sustainable trajectory'

```

```

 },

```

```

 gauge_choice="Hold identity/values constant while state varies"

```

```

)

```

```

 def assess_current_state(

```

```

 self,

```

```

 user_inputs: Dict[str, float],

```

```

 confidence: Dict[str, float]

```

```

) -> Tuple[SystemState, Dict]:

```

```

 """

```

```

 Assess current state from user inputs

```

```

 Returns state estimate + interpretation

```

```

 """

```

```

 # Observe with uncertainty

```

```

 state = self.observer.observe(user_inputs, confidence)

```

```

 # Interpret in reference frame

```

```

 interpretation = self.default_frame.transform_to_frame(state)

```

```

 return state, interpretation

```

```

 def generate_support_plan(

```

```

 self,

```

```

 current_state: SystemState,

```

```

target_state: Optional[SystemState] = None,
horizon: int = 24
) -> Dict:
"""
Generate personalized support plan
Returns prioritized interventions with rationale
"""
Default target: sustainable equilibrium
if target_state is None:
 target_state = SystemState(
 timestamp=current_state.timestamp,
 pressure=5.0,
 margin=7.0,
 temperature=4.0,
 drift=0.0,
 escalation_risk=0.2
)

Compute optimal control
control = self.controller.compute_optimal_control(
 current_state,
 target_state,
 constraints={'max_pressure_relief': 3.0, 'max_capacity_building': 2.0}
)

Select high-leverage interventions
recommendations = []

if control.pressure_relief > 0.5:
 pressure_options = self.interventions.pressure_relief_interventions()
 recommendations.extend([
 **opt, 'priority': 'HIGH', 'rationale': 'System under excessive load'
 for opt in pressure_options[:2]
])

if control.capacity_building > 0.5:
 margin_options = self.interventions.margin_expansion_interventions()
 recommendations.extend([
 **opt, 'priority': 'MEDIUM', 'rationale': 'Low safety margin detected'
 for opt in margin_options[:2]
])

if control.cooling_input > 0.5:
 thermal_options = self.interventions.thermal_regulation_interventions()
 recommendations.extend([
 **opt, 'priority': 'HIGH', 'rationale': 'High internal agitation'
 for opt in thermal_options[:1]
])

Emergency protocol if critical
if current_state.escalation_risk > 0.7:
 emergency = self.controller.emergency_stabilization(current_state)
 recommendations.insert(0, {
 'name': 'EMERGENCY STABILIZATION',
 'priority': 'CRITICAL',
 'rationale': 'System near critical point - immediate intervention required',
 'control': emergency[0]
 })

```

```

 })

 # Predict trajectory
 control_sequence = [control] * horizon
 predicted = self.dynamics.predict_trajectory(
 current_state,
 control_sequence,
 horizon
)

 return {
 'current_state': current_state,
 'target_state': target_state,
 'optimal_control': control,
 'recommendations': recommendations,
 'predicted_trajectory': predicted,
 'key_insights': self._generate_insights(current_state, predicted)
 }

def _generate_insights(
 self,
 current: SystemState,
 trajectory: List[SystemState]
) -> List[str]:
 """Generate key insights about system dynamics"""
 insights = []

 # Trend analysis
 final = trajectory[-1]
 if final.strain < current.strain:
 insights.append(f" Interventions should reduce strain by {current.strain - final.strain:.1f} points")

 # Critical warnings
 if any(s.escalation_risk > 0.8 for s in trajectory):
 insights.append(" Trajectory passes through high-risk zone - consider additional support")

 # Leverage opportunities
 if current.margin > 6 and current.pressure < 5:
 insights.append(" System has capacity - good time for proactive margin expansion")

 return insights

=====
EXAMPLE USAGE
=====

def example_usage():
 """Demonstrate the module in action"""

 print("# Individual Support Module - Example Session\n")

 # Initialize
 support = IndividualSupportModule()

 # Simulated user inputs (would come from conversation/questionnaire)
 user_inputs = {

```

```

 'pressure': 8.5, # High load
 'margin': 2.0, # Low buffer
 'temperature': 7.0, # Agitated
 'drift': 1.5, # Drifting away from equilibrium
 'escalation_risk': 0.65 # Approaching critical point
}

confidence = {
 'pressure': 0.8,
 'margin': 0.6,
 'temperature': 0.7,
 'drift': 0.4,
 'escalation_risk': 0.5
}

print("## Current State Assessment\n")

Assess state
current_state, interpretation = support.assess_current_state(user_inputs, confidence)

print(f"***Timestamp:** {current_state.timestamp}")
print(f"***Overall Strain:** {current_state.strain:.1f}/10\n")

print("### Reference Frame: " + interpretation['frame'])
for insight in interpretation['narrative']:
 print(f"- {insight}")

print("\n## Personalized Support Plan\n")

Generate plan
plan = support.generate_support_plan(current_state, horizon=12)

print("### Recommended Interventions\n")
for rec in plan['recommendations']:
 print(f"***{rec['priority']}**": {rec['name']}")
 print(f"- {rec['description']}")
 print(f"- Leverage: {rec.get('leverage', 'N/A')}")
 print(f"- Rationale: {rec['rationale']}\n")

print("### Key Insights\n")
for insight in plan['key_insights']:
 print(f"- {insight}")

print("\n### Predicted Trajectory (next 12 hours)\n")
traj = plan['predicted_trajectory']
print(f"- Initial strain: {traj[0].strain:.1f}")
print(f"- Final strain: {traj[-1].strain:.1f}")
print(f"- Change: {traj[-1].strain - traj[0].strain:+.1f}\n")

print("---\n")
print(" **Module successfully demonstrated physics-grounded support framework**")

if __name__ == "__main__":
 example_usage()

```

---

## ## Key Features

### ### \*\*Core Capabilities\*\*

- \*\* Physics-Grounded Modeling \*\*: Maps personal experiences to rigorous physical analogues
- \*\* State-Space Dynamics \*\*: Tracks system evolution through multi-dimensional state space
- \*\* Optimal Control \*\*: Computes high-leverage interventions using control theory
- \*\* Uncertainty Handling \*\*: Bayesian state estimation under partial observability
- \*\* Trajectory Prediction \*\*: Forward simulation for intervention planning
- \*\* Witnessed Clarity \*\*: Interpretable reference frames for sense-making
- \*\* Emergency Protocols \*\*: Automatic escalation for critical states

---

## ## Usage Patterns

### ### Quick Assessment

```
python
support = IndividualSupportModule()
state, interpretation = support.assess_current_state(user_inputs, confidence)
```

### ### Generate Support Plan

```
python
plan = support.generate_support_plan(current_state, horizon=24)
```

### ### Emergency Intervention

```
python
emergency_controls = support.controller.emergency_stabilization(critical_state)
```

---

## ## Multilingual Support

The module structure supports **all languages** - simply translate:

- State variable names
- Interpretation narratives
- Intervention descriptions

Core physics remains language-independent!

---

## ## Next Steps

### **Possible extensions:**

1. **Machine learning**: Learn dynamics from real user data
2. **Multi-person systems**: Extend to relationship/team dynamics
3. **Adaptive control**: Controller that learns optimal strategies per person
4. **Mobile interface**: Real-time state tracking + notifications
5. **Integration**: Connect to calendar, messages, biometrics for automatic state estimation

---



**\*\* This module treats personal wellbeing as a rigorous engineering problem while preserving human dignity and agency.\*\***